

# AlgoRecall

## Project Proposal for UCS503P

**Submitted to:** Ms. Paramveer Kaur

**Thapar Institute of Engineering and Technology**

**Pihu Tanoch, 1024170373 | Ekmanjot Kaur, 1024170377 | Mannatpreet Singh, 1024170382**

## 1 Higher-order goal

This project aims to improve long-term retention and consistency in Data Structures and Algorithms (DSA) preparation by reducing the time and effort required to organize and revise previously solved questions. While DSA preparation involves solving a large number of problems across different topics, the immediate engineering objective is to translate systematic revision practices into a measurable, deployable software solution.

## 2 Time-to-value

- We will deliver meaningful increments quickly by prototyping the core workflow of adding solved questions, scheduling revisions, and viewing questions that are due for revision.
- We will prioritize fast feedback over perfection by developing the platform incrementally and testing the revision workflow and analytics during development.
- Success is defined by what can be delivered and measured in the first iteration, then improved based on user feedback and revision activity.

## 3 Problem Statement

Students and job aspirants preparing for technical interviews solve a large number of DSA questions across topics such as Arrays, Linked Lists, Trees, Graphs, Dynamic Programming, and Greedy Algorithms. As the number of solved questions increases, it becomes difficult to remember previously learned concepts, problem-solving patterns, and algorithmic approaches.

Common pain points include:

- **Unstructured revision:** Revisions are often performed irregularly and without a structured plan, making it difficult to determine which questions require attention.
- **Poor retention:** Previously learned concepts and problem-solving patterns may be forgotten when solved questions are not revisited regularly.

- **Lack of prioritization:** Learners may find it difficult to determine which questions should be revised first based on difficulty, confidence, or time since the last revision.
- **Weak topic identification:** Identifying weak topics and areas requiring additional practice becomes difficult as the number of solved questions increases.
- **Fragmented tracking:** Learners often maintain records using spreadsheets, notes, or coding platforms, which provide limited support for systematic revision management.

Why this matters now: As the number of solved DSA questions grows during technical interview preparation, efficient revision becomes increasingly important for maintaining knowledge, improving problem-solving patterns, and preparing consistently.

## 4 Proposed Solution

### 4.1 Overview

Build a web-based “AlgoRecall” system with the following components:

- **Question management:** Users add solved DSA questions with details such as topic, platform, difficulty level, and confidence level.
- **Revision scheduling:** The system schedules future revisions based on revision-related parameters.
- **Due Today dashboard:** Users can view questions that are due for revision.
- **Revision history:** The system maintains records of previous revision activities.
- **Analytics dashboard:** Users can view revision progress, topic-wise performance, confidence distribution, and revision consistency.
- **Search and filter:** Users can quickly retrieve questions based on different criteria.

### 4.2 Core workflow

1. User registers or logs into the platform.
2. User adds a solved DSA question with details such as topic, difficulty, platform, and confidence level.
3. System schedules a future revision for the question.
4. User views questions that are due for revision through the dashboard.
5. User revises a question and updates the confidence level.
6. System records the revision activity and schedules the next revision.
7. User monitors progress and performance through analytics and reports.

### 4.3 Operational constraints for an educational setting

- Keep the solution simple and usable for students preparing for technical interviews.
- Avoid dependence on advanced research algorithms; focus on revision management, scheduling, analytics, and system correctness.
- Design the system for deployment using common web hosting and managed database services.

- Ensure that the platform remains responsive and accessible across commonly used devices.

## 5 Solution Approach

This is an engineering course project, so we focus on scalable revision management and learning analytics rather than novel algorithm research.

### 5.1 Revision scheduling assistance

Revision scheduling will be parameter-based:

- Store question attributes such as topic, difficulty, confidence level, date solved, last revision date, next revision date, and revision count.
- Use Confidence Level, Question Difficulty, and Last Revision Date to determine when a question should be revised next.
- Questions with lower confidence levels, higher difficulty, or longer gaps since the last revision will be given higher priority.
- The system will calculate the next revision date and assign a priority to each question.
- Revision activities will be recorded so that the system can maintain a complete revision history.

### 5.2 Web architecture

A typical 3-tier web architecture:

- **Frontend:** React.js with Tailwind CSS for question management, revision dashboards, status views, and analytics.
- **Backend API:** Node.js and Express.js for authentication, question management, revision scheduling, revision history, and analytics.
- **Database:** MongoDB Atlas to store users, questions, and revision history.

### 5.3 CI/CD and fast delivery enabler

CI/CD will be used to:

- Automatically build and test the application on every change.
- Produce repeatable deployment artifacts.
- Enable safe and frequent releases of incremental features.

## 6 Evaluation Criterion

We define success using measurable metrics tied to the core revision workflow.

### 6.1 Primary evaluation metric

**Revision consistency:** The proportion of scheduled revisions completed by the user within the intended revision period.

- **Target:** Improve the consistency of scheduled revisions during the pilot window.
- **Attribution:** Measured using scheduled revision dates and actual revision timestamps stored in the database.

## 6.2 Secondary evaluation metrics

- **Revision completion rate:** Percentage of questions due for revision that are successfully revised.
- **Confidence improvement:** Change in confidence levels for questions across revision sessions.
- **Topic-wise performance:** Distribution of confidence levels and revision activity across different DSA topics.
- **Revision backlog:** Number of questions that remain overdue for revision.
- **Revision frequency:** Number of revision sessions completed during a given period.
- **User clarity:** User-reported ease of managing and revising questions through the platform.

## 6.3 Pilot validation plan

- Recruit a small group of DSA learners to use the platform during the pilot window.
- Record solved questions, scheduled revisions, completed revisions, and confidence updates.
- Compare revision consistency and revision backlog before and after using the structured workflow.
- Collect user feedback regarding usability, scheduling, search and filtering, and analytics.

# 7 Scalability

The proposition should scale in both theoretical and practical senses.

1. **Scalable (theoretically):** The system should support growth in the number of users, questions, and revision records by:
  - decoupling frontend and backend,
  - enabling pagination and indexing for common queries,
  - using efficient database queries,
  - using stateless REST APIs.
2. **Operationally deployable (practically):** The system should run on common web hosting:
  - managed database,
  - hosted frontend and backend,
  - CI/CD pipeline for deployment,
  - environment-based configuration for different deployment environments.

## 8 Engine availability heuristic

A mature set of “engines” (tooling/services) is available to implement this as a web-based project:

- Web framework for frontend development using React.js.
- Backend framework for REST APIs using Node.js and Express.js.
- Managed database using MongoDB Atlas.
- Database interaction using Mongoose ODM.
- Authentication approach using JSON Web Tokens (JWT).
- Password security using bcrypt hashing.
- Data visualization using Chart.js.
- Version control using Git and GitHub.
- Development environment using Visual Studio Code.

We will use these mature components to focus on revision workflow design, scheduling, analytics, correctness, and usability rather than inventing new infrastructure.

## **9 Project scope and deliverables**

### **9.1 Initial deliverable**

- User registration and login.
- Add, edit, delete, and view solved DSA questions.
- Store question details including topic, platform, difficulty, and confidence level.
- Basic revision scheduling.
- Due Today dashboard.
- Record revision activity.
- Basic search and filtering.
- Database structure for users, questions, and revision history.

### **9.2 Subsequent deliverables**

- Improved revision scheduling based on confidence, difficulty, and revision history.
- Complete revision history interface.
- Analytics dashboard for revision progress and topic-wise performance.
- Confidence distribution visualizations.
- Revision backlog and priority indicators.
- Improved search and filtering.
- Pagination and indexed queries for scale readiness.
- Enhanced authentication and security.
- CI/CD pipeline for staging and production deployment.

## **10 Risks and mitigations**

- **Data privacy / security:** Minimize collected data and protect user credentials using secure authentication and password hashing.
- **User adoption:** Provide an easy interface for adding questions and completing revisions; keep workflow steps minimal.
- **Revision scheduling effectiveness:** Begin with clearly defined scheduling parameters and improve the scheduling process based on user feedback and observed revision activity.
- **Incomplete revision records:** Record revision activity and confidence updates directly through the application workflow.
- **Evaluation measurement ambiguity:** Define clear revision metrics and record timestamps before evaluation begins.
- **Operational issues during deployment:** Use version control and CI/CD to produce repeatable builds and deployments.

## 11 Summary

This project proposes a deployable web platform to improve DSA revision consistency and long-term retention. It meets the course criteria by emphasizing time-to-value through incremental development, implementing structured revision scheduling and analytics, and using measurable evaluation criteria such as revision consistency, revision completion rate, confidence improvement, and revision backlog. It remains focused on engineering a practical revision-management and learning-analytics platform rather than research-driven algorithm development.